

Ejemplo visto en clase, generado con herramientas de IA

Prompt: Sos un desarrollador backend que está realizando un servicio para incorporar datos a una tabla cliente, que contenga los siguientes datos: identificación de cliente, apellido, nombre, identificación del domicilio, fecha de baja. En otra tabla se debe administrar el domicilio del cliente y vincular ambas. debes generar la sentencia de creación para MySQL, tanto de las tablas como de los procedimientos que inserten y /o actualicen esta información. Finalmente crear una api en java para poder utilizar estos procedimientos desde una aplicación.

(Herramienta utilizada DeepSeek)

A continuación se presenta el resultado obtenido de esta pregunta muy básica.

Solución Backend para Gestión de Clientes y Domicilios

1. Estructura de Base de Datos MySQL

Creación de tablas

```
```sql
-- Tabla de Domicilios (primero porque Cliente la referencia)
CREATE TABLE domicilios (
 id_domicilio INT AUTO_INCREMENT PRIMARY KEY,
 calle VARCHAR(100) NOT NULL,
 numero VARCHAR(20) NOT NULL,
 piso VARCHAR(10),
 departamento VARCHAR(10),
 ciudad VARCHAR(50) NOT NULL,
 provincia VARCHAR(50) NOT NULL,
 codigo_postal VARCHAR(20) NOT NULL,
 fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
 CURRENT_TIMESTAMP
);

-- Tabla de Clientes
CREATE TABLE clientes (
 id_cliente INT AUTO_INCREMENT PRIMARY KEY,
 identificacion VARCHAR(20) UNIQUE NOT NULL,
 apellido VARCHAR(50) NOT NULL,
 nombre VARCHAR(50) NOT NULL,
 id_domicilio INT,
 fecha_baja DATE,
 fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
 CURRENT_TIMESTAMP,
 FOREIGN KEY (id_domicilio) REFERENCES domicilios(id_domicilio)
);
```
```

Procedimientos almacenados

```
```sql
-- Procedimiento para insertar domicilio
DELIMITER //
CREATE PROCEDURE sp_insertar_domicilio(
 IN p_calle VARCHAR(100),
 IN p_numero VARCHAR(20),
 IN p_piso VARCHAR(10),
 IN p_departamento VARCHAR(10),
 IN p_ciudad VARCHAR(50),

```

```
 IN p_provincia VARCHAR(50),
 IN p_codigo_postal VARCHAR(20),
 OUT p_id_domicilio INT
)
BEGIN
 INSERT INTO domicilios (calle, numero, piso, departamento, ciudad, provincia,
codigo_postal)
 VALUES (p_calle, p_numero, p_piso, p_departamento, p_ciudad, p_provincia,
p_codigo_postal);

 SET p_id_domicilio = LAST_INSERT_ID();
END //
DELIMITER ;

-- Procedimiento para actualizar domicilio
DELIMITER //
CREATE PROCEDURE sp_actualizar_domicilio(
 IN p_id_domicilio INT,
 IN p_calle VARCHAR(100),
 IN p_numero VARCHAR(20),
 IN p_piso VARCHAR(10),
 IN p_departamento VARCHAR(10),
 IN p_ciudad VARCHAR(50),
 IN p_provincia VARCHAR(50),
 IN p_codigo_postal VARCHAR(20)
)
BEGIN
 UPDATE domicilios
 SET calle = p_calle,
 numero = p_numero,
 piso = p_piso,
 departamento = p_departamento,
 ciudad = p_ciudad,
 provincia = p_provincia,
 codigo_postal = p_codigo_postal
 WHERE id_domicilio = p_id_domicilio;
END //
DELIMITER ;

-- Procedimiento para insertar cliente
DELIMITER //
CREATE PROCEDURE sp_insertar_cliente(
 IN p_identificacion VARCHAR(20),
 IN p_apellido VARCHAR(50),
 IN p_nombre VARCHAR(50),
 IN p_id_domicilio INT,
 OUT p_id_cliente INT
)
)
```

```

BEGIN
 INSERT INTO clientes (identificacion, apellido, nombre, id_domicilio)
 VALUES (p_identificacion, p_apellido, p_nombre, p_id_domicilio);

 SET p_id_cliente = LAST_INSERT_ID();
END //
DELIMITER ;

-- Procedimiento para actualizar cliente
DELIMITER //
CREATE PROCEDURE sp_actualizar_cliente(
 IN p_id_cliente INT,
 IN p_identificacion VARCHAR(20),
 IN p_apellido VARCHAR(50),
 IN p_nombre VARCHAR(50),
 IN p_id_domicilio INT
)
BEGIN
 UPDATE clientes
 SET identificacion = p_identificacion,
 apellido = p_apellido,
 nombre = p_nombre,
 id_domicilio = p_id_domicilio
 WHERE id_cliente = p_id_cliente;
END //
DELIMITER ;

-- Procedimiento para dar de baja un cliente
DELIMITER //
CREATE PROCEDURE sp_baja_cliente(
 IN p_id_cliente INT,
 IN p_fecha_baja DATE
)
BEGIN
 UPDATE clientes
 SET fecha_baja = p_fecha_baja
 WHERE id_cliente = p_id_cliente;
END //
DELIMITER ;

```

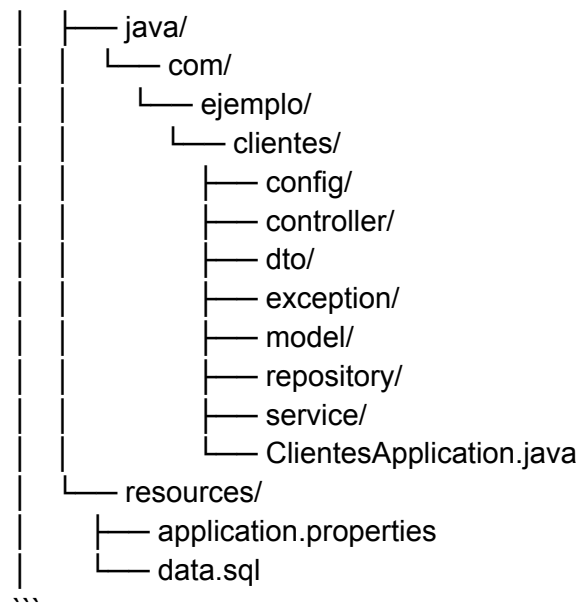
## ## 2. API Java con Spring Boot

### ### Estructura del proyecto

```

...
src/
└── main/

```



### ### Dependencias Maven (pom.xml)

```

'''xml
<dependencies>
 <!-- Spring Boot Starter Web -->
 <dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
 </dependency>

 <!-- Spring Boot Starter Data JPA -->
 <dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-data-jpa</artifactId>
 </dependency>

 <!-- MySQL Connector -->
 <dependency>
 <groupId>mysql</groupId>
 <artifactId>mysql-connector-java</artifactId>
 <scope>runtime</scope>
 </dependency>

 <!-- Lombok -->
 <dependency>
 <groupId>org.projectlombok</groupId>
 <artifactId>lombok</artifactId>
 <optional>true</optional>
 </dependency>
</dependencies>
'''

```

```
application.properties
```

```
``properties
spring.datasource.url=jdbc:mysql://localhost:3306/nombre_basedatos?useSSL=false&server
Timezone=UTC
spring.datasource.username=usuario
spring.datasource.password=contraseña
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
``
```

```
Modelos
```

```
``java
// Cliente.java
package com.ejemplo.clientes.model;

import jakarta.persistence.*;
import lombok.Data;
import java.util.Date;

@Data
@Entity
@Table(name = "clientes")
public class Cliente {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long idCliente;

 @Column(unique = true, nullable = false)
 private String identificacion;

 @Column(nullable = false)
 private String apellido;

 @Column(nullable = false)
 private String nombre;

 @ManyToOne
 @JoinColumn(name = "id_domicilio")
 private Domicilio domicilio;

 private Date fechaBaja;
}
```

```
@Column(uptable = false)
private Date fechaCreacion;

private Date fechaActualizacion;

@PrePersist
protected void onCreate() {
 fechaCreacion = new Date();
 fechaActualizacion = new Date();
}

@PreUpdate
protected void onUpdate() {
 fechaActualizacion = new Date();
}
}

// Domicilio.java
package com.ejemplo.clientes.model;

import jakarta.persistence.*;
import lombok.Data;
import java.util.Date;

@Data
@Entity
@Table(name = "domicilios")
public class Domicilio {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long idDomicilio;

 @Column(nullable = false)
 private String calle;

 @Column(nullable = false)
 private String numero;

 private String piso;
 private String departamento;

 @Column(nullable = false)
 private String ciudad;

 @Column(nullable = false)
 private String provincia;

 @Column(nullable = false)
```

```
private String codigoPostal;

@Column(uptable = false)
private Date fechaCreacion;

private Date fechaActualizacion;

@PrePersist
protected void onCreate() {
 fechaCreacion = new Date();
 fechaActualizacion = new Date();
}

@PreUpdate
protected void onUpdate() {
 fechaActualizacion = new Date();
}
}
...
```

### ### DTOs

```
```java
// ClienteDTO.java
package com.ejemplo.clientes.dto;

import lombok.Data;
import java.util.Date;
```

```
@Data
public class ClienteDTO {
    private Long idCliente;
    private String identificacion;
    private String apellido;
    private String nombre;
    private DomicilioDTO domicilio;
    private Date fechaBaja;
}
```

```
// DomicilioDTO.java
package com.ejemplo.clientes.dto;
```

```
import lombok.Data;
```

```
@Data
public class DomicilioDTO {
    private Long idDomicilio;
    private String calle;
```



```
private String numero;  
private String piso;  
private String departamento;  
private String ciudad;  
private String provincia;  
private String codigoPostal;  
}  
...
```

Repositorios

```
```java  
// ClienteRepository.java
package com.ejemplo.clientes.repository;

import com.ejemplo.clientes.model.Cliente;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.query.Procedure;
import org.springframework.data.repository.query.Param;

public interface ClienteRepository extends JpaRepository<Cliente, Long> {
 @Procedure(procedureName = "sp_insertar_cliente")
 Long insertarCliente(
 @Param("p_identificacion") String identificacion,
 @Param("p_apellido") String apellido,
 @Param("p_nombre") String nombre,
 @Param("p_id_domicilio") Long idDomicilio
);

 @Procedure(procedureName = "sp_actualizar_cliente")
 void actualizarCliente(
 @Param("p_id_cliente") Long idCliente,
 @Param("p_identificacion") String identificacion,
 @Param("p_apellido") String apellido,
 @Param("p_nombre") String nombre,
 @Param("p_id_domicilio") Long idDomicilio
);

 @Procedure(procedureName = "sp_baja_cliente")
 void darBajaCliente(
 @Param("p_id_cliente") Long idCliente,
 @Param("p_fecha_baja") Date fechaBaja
);
}

// DomicilioRepository.java
package com.ejemplo.clientes.repository;
```

```

import com.ejemplo.clientes.model.Domicilio;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.query.Procedure;
import org.springframework.data.repository.query.Param;

public interface DomicilioRepository extends JpaRepository<Domicilio, Long> {
 @Procedure(procedureName = "sp_insertar_domicilio")
 Long insertarDomicilio(
 @Param("p_calle") String calle,
 @Param("p_numero") String numero,
 @Param("p_piso") String piso,
 @Param("p_departamento") String departamento,
 @Param("p_ciudad") String ciudad,
 @Param("p_provincia") String provincia,
 @Param("p_codigo_postal") String codigoPostal
);

 @Procedure(procedureName = "sp_actualizar_domicilio")
 void actualizarDomicilio(
 @Param("p_id_domicilio") Long idDomicilio,
 @Param("p_calle") String calle,
 @Param("p_numero") String numero,
 @Param("p_piso") String piso,
 @Param("p_departamento") String departamento,
 @Param("p_ciudad") String ciudad,
 @Param("p_provincia") String provincia,
 @Param("p_codigo_postal") String codigoPostal
);
}
...

```

### ### Servicios

```

```java
// ClienteService.java
package com.ejemplo.clientes.service;

import com.ejemplo.clientes.dto.ClienteDTO;
import com.ejemplo.clientes.dto.DomicilioDTO;
import com.ejemplo.clientes.exception.ResourceNotFoundException;
import com.ejemplo.clientes.repository.ClienteRepository;
import com.ejemplo.clientes.repository.DomicilioRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Date;

```

@Service

```
public class ClienteService {  
    private final ClienteRepository clienteRepository;  
    private final DomicilioRepository domicilioRepository;
```

@Autowired

```
    public ClienteService(ClienteRepository clienteRepository, DomicilioRepository  
domicilioRepository) {  
        this.clienteRepository = clienteRepository;  
        this.domicilioRepository = domicilioRepository;  
    }  
}
```

@Transactional

```
public Long crearCliente(ClienteDTO clienteDTO) {  
    // Primero crear el domicilio  
    DomicilioDTO domicilioDTO = clienteDTO.getDomicilio();  
    Long idDomicilio = domicilioRepository.insertarDomicilio(  
        domicilioDTO.getCalle(),  
        domicilioDTO.getNumero(),  
        domicilioDTO.getPiso(),  
        domicilioDTO.getDepartamento(),  
        domicilioDTO.getCiudad(),  
        domicilioDTO.getProvincia(),  
        domicilioDTO.getCodigoPostal()  
    );  
  
    // Luego crear el cliente  
    return clienteRepository.insertarCliente(  
        clienteDTO.getIdentificacion(),  
        clienteDTO.getApellido(),  
        clienteDTO.getNombre(),  
        idDomicilio  
    );  
}
```

@Transactional

```
public void actualizarCliente(Long idCliente, ClienteDTO clienteDTO) {  
    // Actualizar domicilio  
    DomicilioDTO domicilioDTO = clienteDTO.getDomicilio();  
    domicilioRepository.actualizarDomicilio(  
        domicilioDTO.getIdDomicilio(),  
        domicilioDTO.getCalle(),  
        domicilioDTO.getNumero(),  
        domicilioDTO.getPiso(),  
        domicilioDTO.getDepartamento(),  
        domicilioDTO.getCiudad(),  
        domicilioDTO.getProvincia(),  
        domicilioDTO.getCodigoPostal()  
    );  
}
```

```

    );

    // Actualizar cliente
    clienteRepository.actualizarCliente(
        idCliente,
        clienteDTO.getIdentificacion(),
        clienteDTO.getApellido(),
        clienteDTO.getNombre(),
        domicilioDTO.getIdDomicilio()
    );
}

@Transactional
public void darBajaCliente(Long idCliente, Date fechaBaja) {
    clienteRepository.darBajaCliente(idCliente, fechaBaja);
}
}
...

```

Controladores

```

```java
// ClienteController.java
package com.ejemplo.clientes.controller;

import com.ejemplo.clientes.dto.ClienteDTO;
import com.ejemplo.clientes.service.ClienteService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.Date;

@RestController
@RequestMapping("/api/clientes")
public class ClienteController {
 private final ClienteService clienteService;

 @Autowired
 public ClienteController(ClienteService clienteService) {
 this.clienteService = clienteService;
 }

 @PostMapping
 public ResponseEntity<?> crearCliente(@RequestBody ClienteDTO clienteDTO) {
 Long idCliente = clienteService.crearCliente(clienteDTO);
 }
}

```

```

 URI location = ServletUriComponentsBuilder
 .fromCurrentRequest()
 .path("/{id}")
 .buildAndExpand(idCliente)
 .toUri();

 return ResponseEntity.created(location).build();
 }

 @PutMapping("/{id}")
 public ResponseEntity<?> actualizarCliente(
 @PathVariable Long id,
 @RequestBody ClienteDTO clienteDTO
) {
 clienteService.actualizarCliente(id, clienteDTO);
 return ResponseEntity.noContent().build();
 }

 @PostMapping("/{id}/baja")
 public ResponseEntity<?> darBajaCliente(@PathVariable Long id) {
 clienteService.darBajaCliente(id, new Date());
 return ResponseEntity.noContent().build();
 }
}
...

```

### Clase principal de la aplicación

```

```java
package com.ejemplo.clientes;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ClientesApplication {
    public static void main(String[] args) {
        SpringApplication.run(ClientesApplication.class, args);
    }
}
...

```

3. Uso de la API

Crear un cliente

...

POST /api/clientes

Content-Type: application/json

```
{
  "identificacion": "12345678",
  "apellido": "Pérez",
  "nombre": "Juan",
  "domicilio": {
    "calle": "Av. Siempre Viva",
    "numero": "742",
    "piso": "3",
    "departamento": "A",
    "ciudad": "Springfield",
    "provincia": "Springfield",
    "codigoPostal": "1234"
  }
}
...
```

Actualizar un cliente

...

PUT /api/clientes/1

Content-Type: application/json

```
{
  "identificacion": "12345678",
  "apellido": "Pérez Gómez",
  "nombre": "Juan Carlos",
  "domicilio": {
    "idDomicilio": 1,
    "calle": "Av. Siempre Viva",
    "numero": "742",
    "piso": "3",
    "departamento": "B",
    "ciudad": "Springfield",
    "provincia": "Springfield",
    "codigoPostal": "1234"
  }
}
...
```

Dar de baja un cliente

...

POST /api/clientes/1/baja

...

Esta solución proporciona una base completa para gestionar clientes y sus domicilios, con:

1. Estructura de base de datos relacional en MySQL
2. Procedimientos almacenados para las operaciones CRUD

3. API RESTful en Java con Spring Boot
4. Manejo de transacciones
5. Validaciones básicas

Puedes expandir agregando más validaciones, manejo de errores, paginación, búsquedas, etc. según tus necesidades específicas.